

Debunkr

A Filipino Fake News Classifier

A Presentation by:

Carado, Deocampo, Encanto, Falco, Torre

According to Pulse Asia Survey,

86% Filipino adults see fake news as a serious problem.

90% Encountered false political information, mainly via social media and TV.

65% Find it hard to identify fake news.

55% Experience frequent exposure to misinformation.





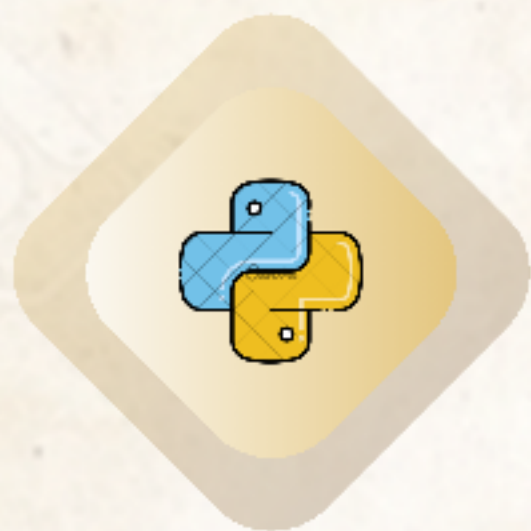
- **Misinformation** weakens public discourse, trust in democracy, and informed decision-making.
- **AI-generated disinformation** (deepfakes, manipulated content) intensifies the issue.
- Few accessible tools exist for Tagalog, leaving Filipinos **vulnerable to misinformation.**

PURPOSE

Our goal is to foster a trustworthy media environment by combating fake news in the Philippines, especially during the May 2025 midterm elections. We offer a **Tagalog fake news classifier** to help users *identify misinformation, support informed decisions, and protect democracy.*

TECHNOLOGICAL STACK

Overview of the development stack used to develop this project



Python



SvelteKit



TailwindCSS Vite



FastAPI & Uvicorn

FUNCTIONAL

FEATURES



Drag & Drop Files

Upload PDF for analysis



Paste Web Links

Web scraping and prediction



Plain Text Input

Direct Input for fake news detection



LIME Visualization

Shows word influence on model prediction

KEY LIBRARIES USED

- NLTK (3.9.1)
- scikit-learn (1.6.1)
- Trafilatura (2.0.0)
- lime (0.2.0.1)
- python-multipart (0.0.20)
- PyPDF2 (3.0.1)
- calamanCy (0.2.2)
- asyncio (3.4.3)
- chart.js (frontend) (4.4.9)

MODEL DEVELOPMENT



Dataset

Two validated Tagalog and Filipino news datasets (25,664 articles total)



Preprocessing

Normalization, tokenization, stopword removal, lemmatization (Tagalog via stopwords-tl & calamanCy; English via NLTK tools)



Vectorization

TF-IDF converts text into numerical features emphasizing important words.

MODEL

DEVELOPMENT



Models Trained

Logistic Regression, Multinomial Naive Bayes, SGD Classifier, Random Forest



Ensemble Method

VotingClassifier with soft voting for robust predictions

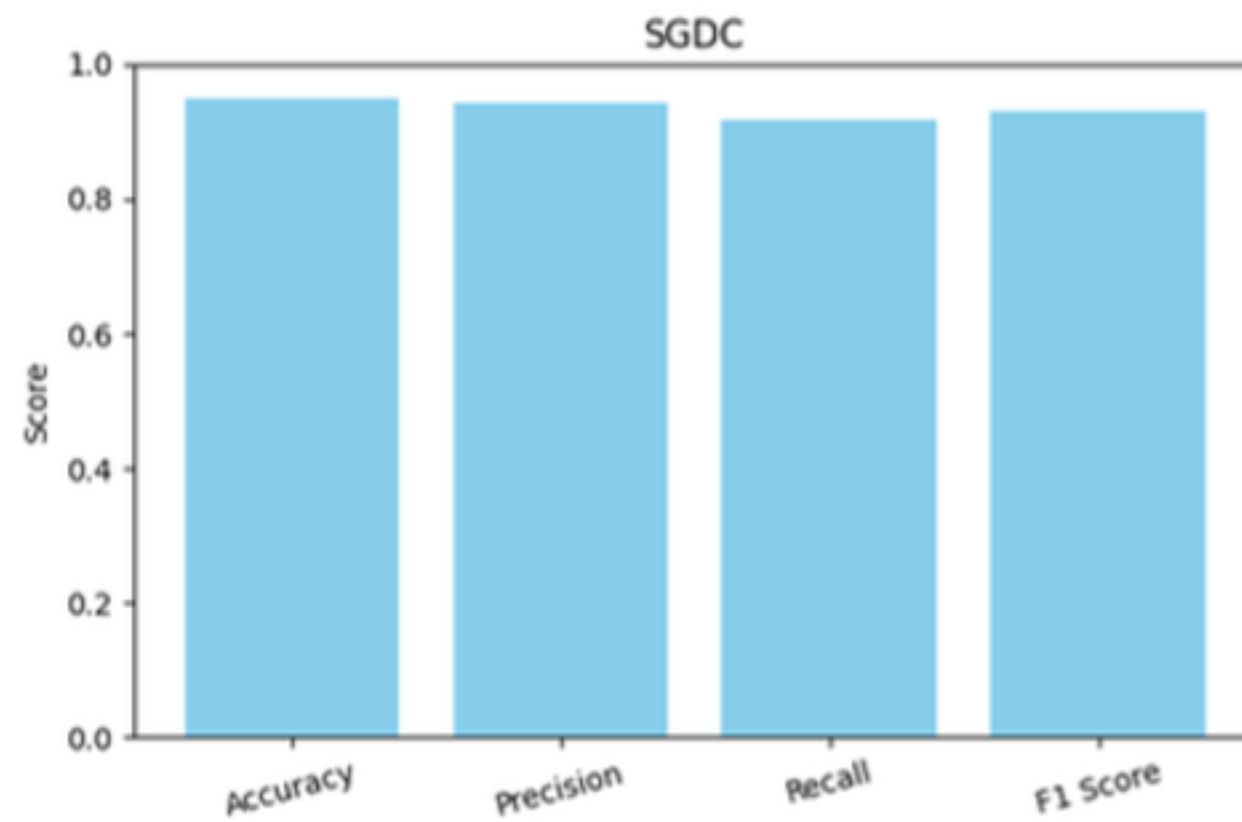
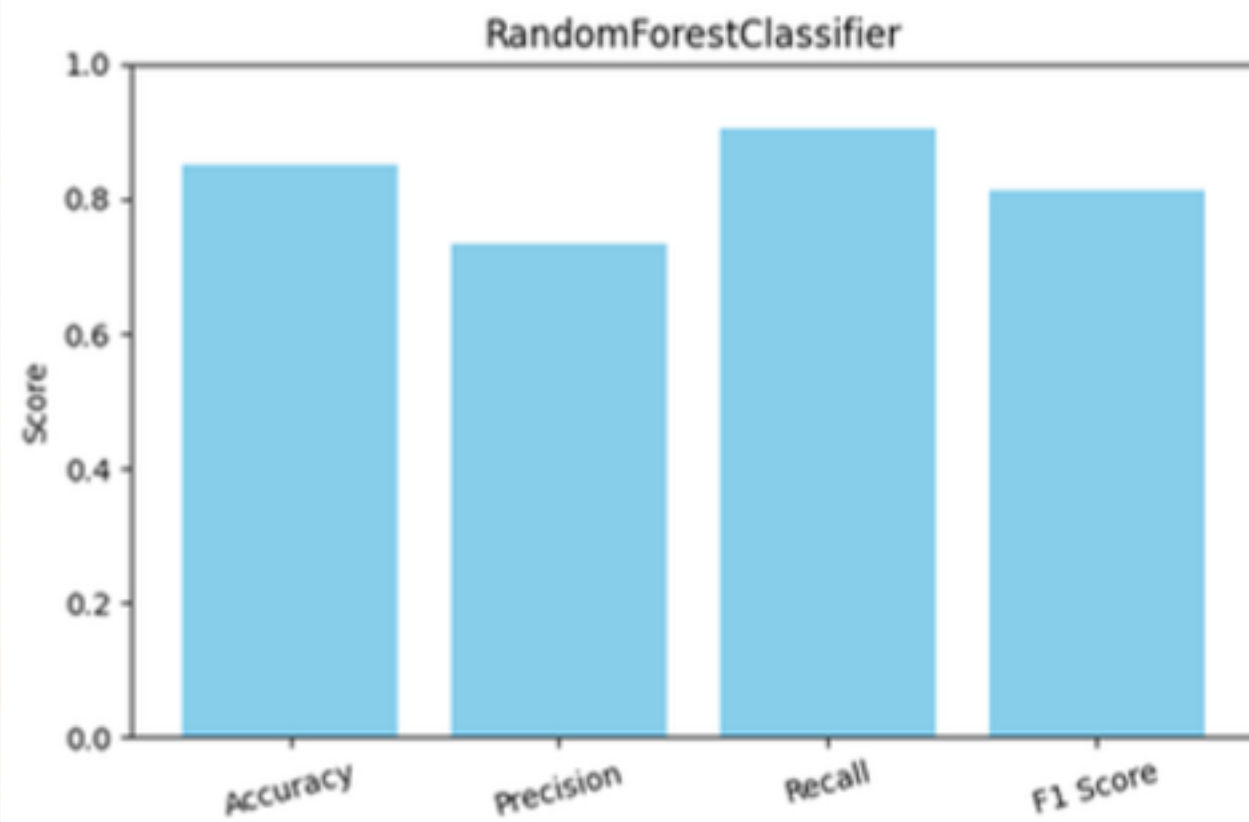
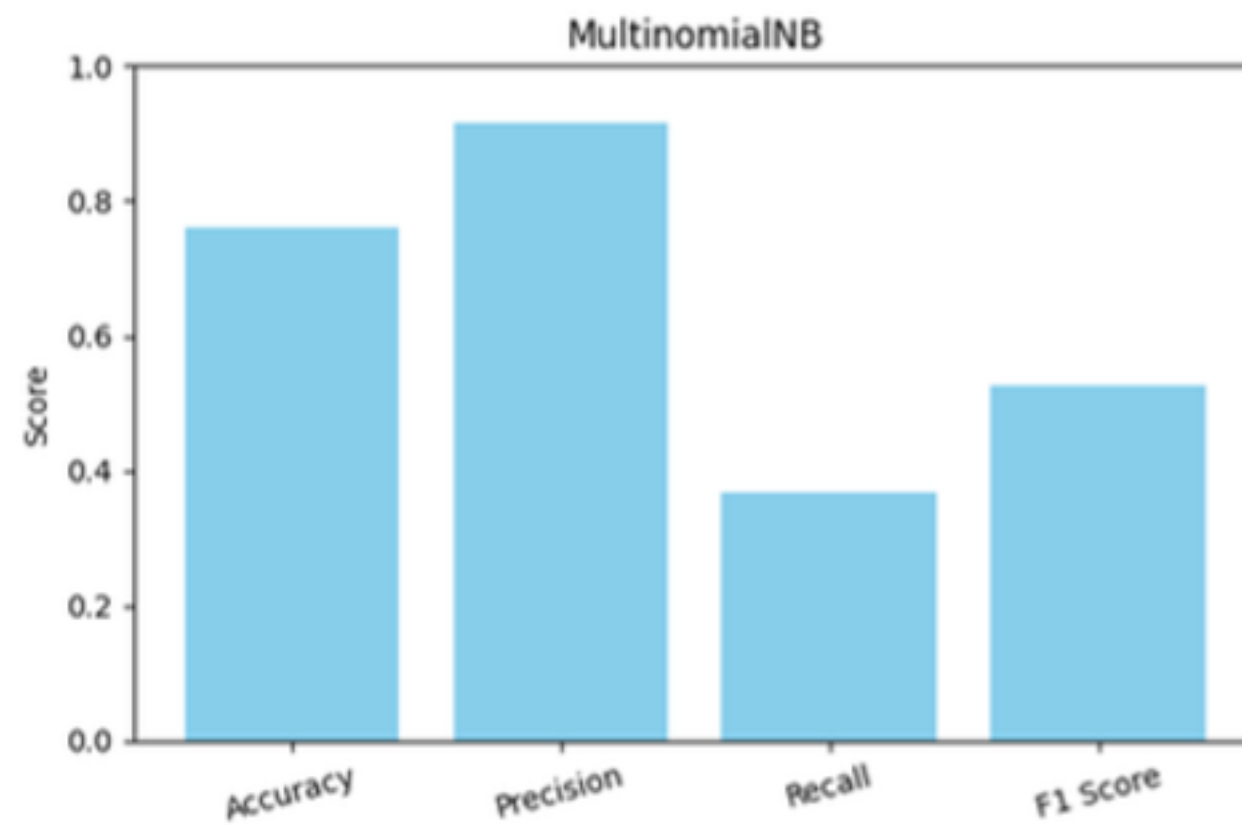
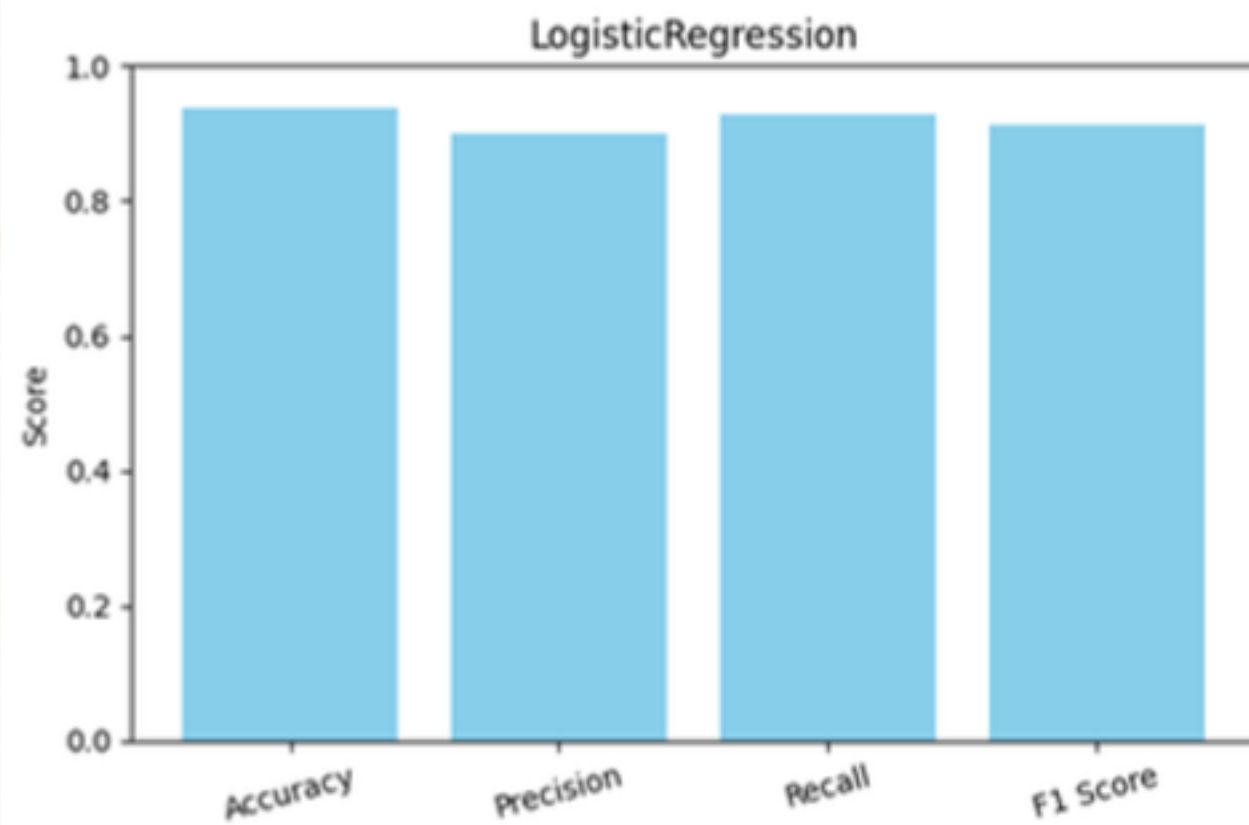


Model Evaluation

Evaluated using accuracy, precision, recall, F1-score, and confusion matrix to measure overall performance and prediction quality.

RESULTS

Result comparison of trained NLP models.



FRONT-END DEVELOPMENT



UI Design

Designed a basic Figma layout to guide UI development.



Front-End Framework

Built with SvelteKit for speed and responsiveness.



Styling

Used TailwindCSS for flexibility and robust community support.



Data Visualization

Used Chart.js to display keywords influencing model predictions.

BACK-END DEVELOPMENT



Server-Side Framework

FastAPI with Uvicorn for efficient front-end communication.



Optimization

Asyncio speeds up responses & FastAPI lifespan caches models to save resources.



Text Extraction Tools

Used Trafilatura for web scraping, pattern matching to clean text, and python-multipart with PyPDF2 for PDF extraction.

LIMITATIONS

- **Outdated Datasets:**

- Tagalog dataset released in 2020; English dataset covers articles from 2016 to 2018.
- Temporal gaps may cause inaccuracies due to the evolution of language and missing recent topics or trends.

- **Limited Tagalog NLP Support:**

- Widely used NLP libraries (e.g., NLTK and spaCy) lack robust support for Tagalog.
- As a low-resource language, Tagalog has limited linguistic resources, making text preprocessing more complex.
- Custom solutions or less mature libraries are required, increasing development time and complexity.

IMPROVEMENTS

Dataset Expansion:

- Include more recent articles (2021–2025) to address outdated data issues.
- Incorporate user-submitted examples for diverse and real-time language representation.

Code-Switching Support:

- Fine-tune the model to handle Tagalog-English code-switching effectively.
- Use techniques like transfer learning from multilingual models or multilingual embeddings.

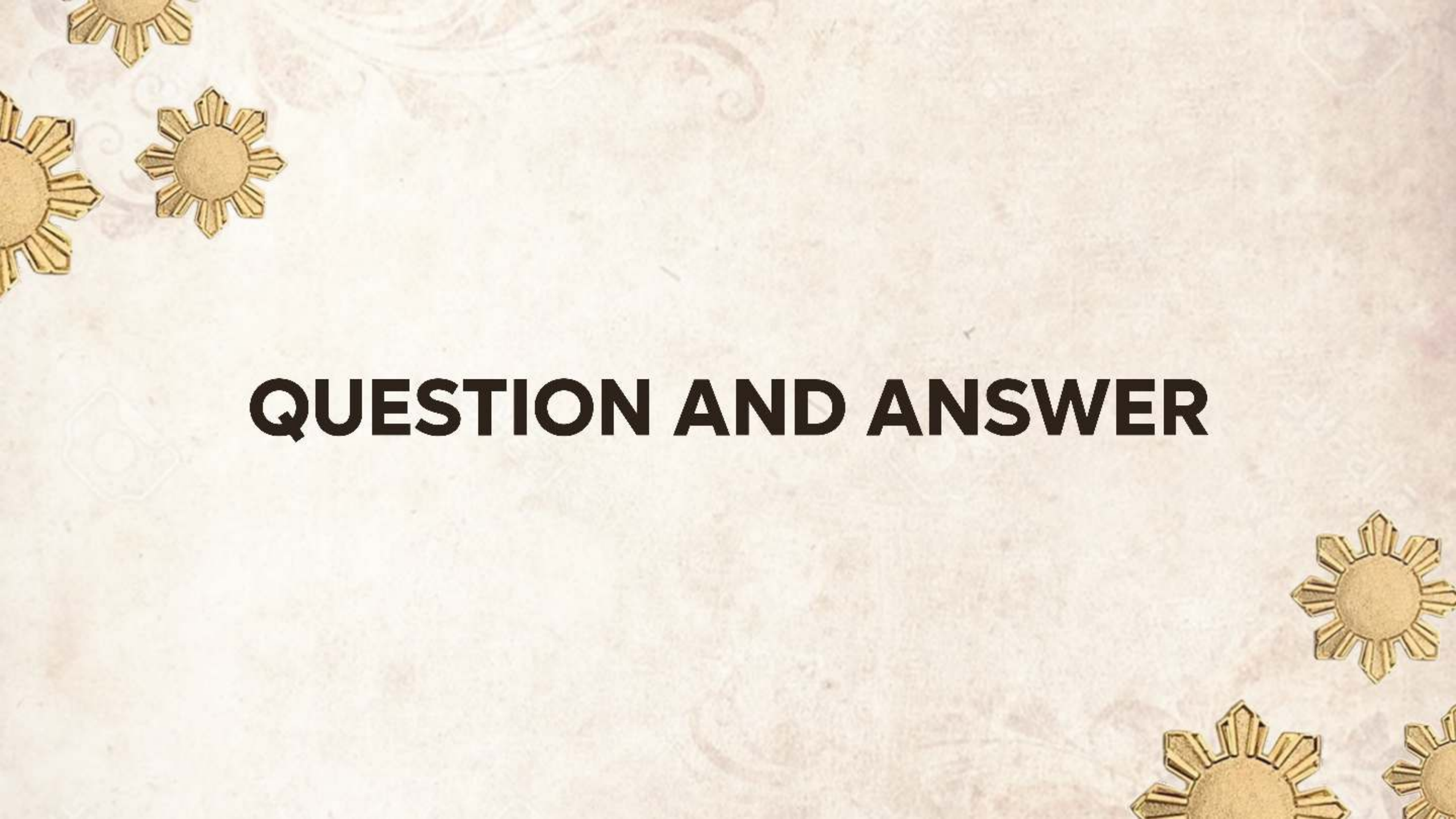
Scalability and Performance Optimization:

- Implement cloud-based scaling solutions (e.g., AWS Lambda, Kubernetes) to manage traffic spikes.
- Dynamically adjust resources for improved scalability and efficient utilization.



WEBSITE DEMONSTRATION

FIN.



QUESTION AND ANSWER